

Brainstorming 與 Deep Research 類 Skill 生態研究

從現行產品、方法論到可抽取 Prompt Pipeline 的落地報告

面向對象：希望在網頁版 ChatGPT Pro、Codex、Copilot 或其他 agent 環境中，將「研究—發想—批判—封裝」做成可重複運行流程的使用者。

版本日期：2026-04-13

報告目的

本報告的核心目標不是列出一串 brainstorming 名詞，而是回答三個落地問題。第一，現行知名的 brainstorming / deep research 類 skill 到底可分成哪些層次。第二，哪些方法最適合逼出真正的新發想，而不只是把舊方法換名字。第三，能否把它們抽取為可讓 ChatGPT Pro 或 Codex 反覆執行的 prompt pipeline 與 skill artifact。

閱讀建議

區段	重點
第 0 章	執行摘要：直接給出結論與優先順序。
第 1 章	把「skill」拆成三層：研究引擎、流程封裝、發想算子。
第 2 章	整理現行主流 deep research / agent skill 生態，區分各自擅長之處。
第 3 章	評估哪些 brainstorming 方法最容易產生新方向。
第 4 章	從現行系統抽取可移植的 prompt architecture。
第 5 章	針對 ChatGPT Pro 的實作建議。
第 6 章	針對 Codex / Copilot / Claude Code 的封裝建議。
第 7 章	給 OrionVault / Obsidian 式記憶層的建議結構。
附錄	直接可用的 prompt、`AGENTS.md`、`SKILL.md` 與 prompt file 模板。

表 0-1 說明：這張表不是形式上的目錄，而是閱讀策略。若你只想立即在 ChatGPT Pro 試做，優先看第 4 至第 5 章與附錄 A。若你要把流程固化給 Codex 或其他 agent，則第 6 章與附錄 C、附錄 D 最重要。

第 0 章 執行摘要

本章先給結論，後面各章再展開理由與做法。

結論編號	核心判斷	對你的直接意義
0-1	今天被稱為「skill」的東西其實至少有三層：研究引擎、流程封裝 artifact、發想算子。	不要把所有問題都丟給單一 prompt；要把研究、發想與封裝分開設計。
0-2	真正決定新穎度的往往不是 deep research 引擎本身，而是後段的 ideation operator。	Deep Research 負責把證據空間掃乾淨；Morphological Analysis、TRIZ、Lateral Thinking 等方法負責逼出新方向。
0-3	對個人版網頁 ChatGPT Pro，最穩的做法是用「Projects + Deep Research + 一組固定 pipeline prompts」來模擬 skill。	你可以先在 ChatGPT Pro 完成研究與發想，再把穩定流程升級成 `AGENTS.md`、`.prompt.md`、`SKILL.md`。[S1][S2][S3][S4]
0-4	若要給 Codex / Copilot / Claude Code 重複使用，抽取的重點不是單一神奇 prompt，而是 prompt architecture。	應把流程拆成 framing、planning、landscape、tension mining、ideation、critique、packaging 七段。[S5][S6][S7][S8][S17][S18]
0-5	在所有常見 brainstorming 方法中，最值得優先放進 agent pipeline 的通常是 Morphological Analysis、TRIZ、Lateral Thinking／Worst Possible Idea。	這三類方法最能迫使模型跳出熟悉答案空間；HMW 與 Six Thinking Hats 更適合做 framing 與補盲。[S19][S20][S21][S22][S24][S25][S26]

表 0-2 說明：這張表濃縮了整份報告最重要的五個判斷。尤其值得注意的是 0-2 與 0-4：如果只把 deep research 當成更長的摘要器，通常不會得到真正新的 idea；要有意識地在研究之後接上結構化發想與惡意審稿兩段。

一句話總結
本報告最重要的一句話是：deep research 負責「找全、找準、找證據」，brainstorming skill 則必須負責「改寫問題、重組維度、製造矛盾、打掉假創新」。因此，面向 agent 的最佳抽取單位不是一個大 prompt，而是一條可回收、可重跑、可批判的工作流。

第 1 章 重新定義你在找的「skill」

若不先澄清「skill」的層次，後面的比較很容易失焦。因為市面上同一個詞，實際上在講三件不同的事。

層次	核心問題	代表例子	對你最有用的地方
研究引擎	如何蒐集、篩選、綜合大量來源，並產出可追溯報告。	ChatGPT Deep Research、Claude Research、Gemini Deep Research、Perplexity Advanced Deep Research。	快速建立 field map，掌握來源、盲點、爭議與證據邊界。 [S1][S10][S11][S13][S14][S15]

流程封裝 artifact	如何把有效流程固化成 always-on 規則、可重用 prompt 或按需載入的 skill。	Projects、`AGENTS.md`、`.prompt.md`、custom agent、subagent、`SKILL.md`。	把成功的互動升級為可持續使用的工作流，而不是每次從頭聊天。[S2][S4][S5][S6][S8][S12][S16][S17][S18]
發想算子	如何迫使思考跳出慣性路徑，生成具有新穎度的方向。	HMW、SCAMPER、Morphological Analysis、TRIZ、Six Thinking Hats、Lateral Thinking、Brainwriting、Worst Possible Idea。	把研究結果轉成新發想，並避免只有換皮的 idea。[S19][S20][S21][S22][S23][S24][S25]

表 1-1 說明：這張表的重要性在於把名詞拆乾淨。多數人把 Deep Research 當成 brainstorming 本身，但它更像是前端掃描器；真正讓你得到『以前沒想過』的，通常是後段發想算子。另一方面，agent 世界裡的 skill 更多是『流程封裝』而不是『思考方法』。三者混在一起時，系統就會同時不夠準、也不夠新。

因此，後續的研究與設計應依序回答三個問題。第一，哪個研究引擎最適合控制來源與產出帶引用報告。第二，哪些發想算子最適合製造高新穎度方向。第三，哪種封裝 artifact 最適合你的實際使用情境：個人網頁版 ChatGPT Pro，或 Codex / Copilot / Claude Code 這類 agent 環境。

第 2 章 現行知名 deep research / agent skill 生態

本章先看研究引擎，再看流程封裝介面。這兩者都很重要，但作用點不同。

2.1 研究引擎：誰負責把證據空間掃乾淨

系統	官方明確特性	對新發想的直接幫助	最適用情境
ChatGPT Deep Research	可指定或限制網站、上傳檔案、可連接 app；先給 research plan，使用者可修改；執行中可追蹤與中斷；最後輸出附 citations 的結構化報告，且可下載成 Markdown、Word、PDF。[S1]	對『控制來源』與『把研究過程做成可驗證報告』非常強。新發想的價值在於它先把場域地圖與矛盾找齊。	你已有明確主題，且希望把網頁、檔案與指定網站一起納入。
Claude Research	Research 以 agentic 方式做多次搜尋並回傳 citations；Anthropic 公開了其多 agent research system 的設計 heuristic，例如 orchestrator-worker、start wide then narrow、parallel subagents 與 judge rubric。[S10][S11]	對『研究流程 heuristics』最有啟發性，特別適合抽 prompt 架構。	你想借用官方公開的 research agent prompting 原則。
Gemini Deep Research	支援來源選擇、研究計畫編修、與 Google 生態來源整合；API 層也提供 Deep Research Agent 的多步研究框架。[S13][S14]	對『以簡單 prompt 啟動，再逐步修改 research plan』很實用。	你有 Gmail／Drive 等來源，或想研究 API 層代理設計。

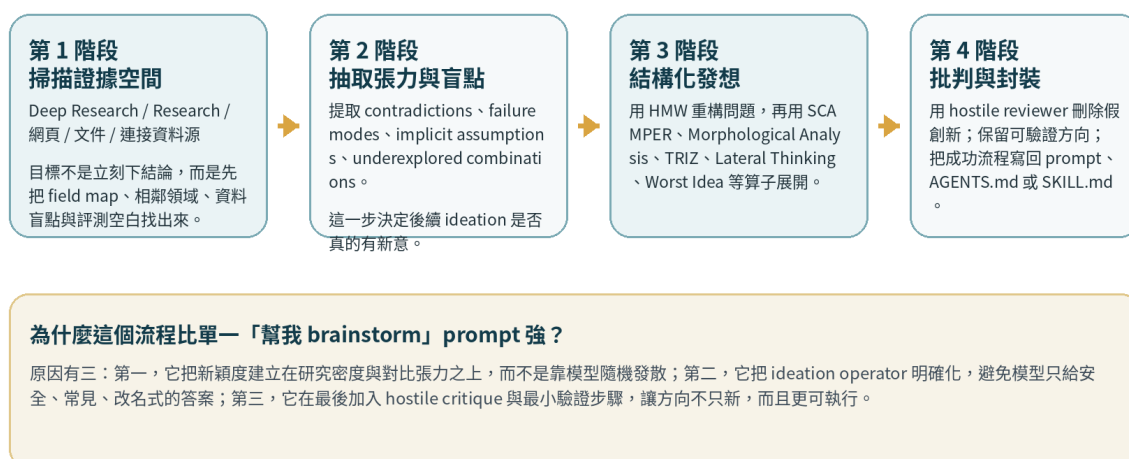
Perplexity Advanced Deep Research	強調更廣的來源搜尋、交叉比對、可追蹤進度、可在執行中 follow-up、可編輯與分享報告。[S15]	對『高召回、快速掃描』很有價值，可作為找相鄰領域訊號的補充。	你想先求 coverage，再決定要不要回到更可控的工作流。
-----------------------------------	---	--------------------------------	--------------------------------

表 2-1 說明：這張表刻意不把系統排名成『誰最好』，因為四者擅長的事情不同。對你的情境，最重要的不是誰回覆最漂亮，而是誰最容易被拆成可重用 workflow。就這點來看，ChatGPT Deep Research 與 Claude Research 的價值最大：前者把來源控制、research plan、下載格式與報告視圖做成清楚介面；後者則把可學習的 heuristics 公開得最明白。

一個關鍵觀察是：這些研究引擎本身都不是 novelty engine。它們最多只能為 novelty 提供素材與結構。真正的新方向，仍需要後段的 ideation operator 去故意重組維度、翻轉假設或處理矛盾。這也是為什麼只靠『請幫我 brainstorm』通常只能得到合理但平庸的答案。

從深度研究到新發想：可重用技能的推薦流程

核心觀點：新的想法通常不是由單一步驟產生，而是由「蒐證 → 張力抽取 → 結構化發想 → 批判封裝」連續流程產生。



Evidence-first Tension mining Structured ideation Adversarial review Reusable skill

圖 2-1 從證據空間到可封裝 skill 的完整 workflow

圖 2-1 說明：這張流程圖把『研究』與『發想』分成四個明確階段。第一段是掃描證據空間，核心任務是建立 field map、找出主要子題、資料來源、基準與代表性工作。第二段不是立刻發想，而是先抽取 tensions、contradictions、evaluation blind spots 與 deployment constraints，因為真正的新方向往往藏在這些張力裡。第三段才進入結構化發想，使用 SCAMPER、Morphological Analysis、TRIZ、Lateral Thinking 等算子把張力轉成 idea seed。第四段則不是收尾，而是另一個關鍵階段：必須加入 hostile critique，把只換名字、只做參數微調或只做淺層遷移的想法刪掉，然後把有效流程封裝為可重用 artifact。這張圖的重點不是視覺漂亮，而是提醒你不要把 research 與 ideation 混成一步。

2.2 流程封裝：真正的 skill 不是長 prompt，而是可回收的 artifact

介面或 artifact	定位	何時使用	對你最重要的含義
Projects (ChatGPT)	將聊天、檔案、指令與記憶聚到同一工作空間；可用 project-only memory 控制跨專案污染。[S2]	你想在網頁版持續累積上下文、文件與研究成果時。	Project 可視為記憶層，不等於完整 skill，但足以當個人版 skill 的基底。

Skills (ChatGPT / Codex / API)	可重用、可分享的工作流；官方將其視為 open standard 的 skill artifact。[S4][S5][S6]	你要把成功流程封裝成可按需載入的能力時。	對個人版 Pro，最保守做法是先用 Project 模擬；對 Codex 則可直接做 `SKILL.md`。
`AGENTS.md`	repo 級 always-on 行為說明，Codex 會在工作前讀取。[S8]	你需要統一專案結構、命名、驗證標準與何謂 trivial idea。	這是長期規則層，不適合放單次任務 prompt。
`prompt.md` / prompt files	單一責任、可重用的 prompt 模板；VS Code 亦支援從有效對話抽取 reusable prompt。[S17]	你已知流程中的某一段會反覆使用。	最適合把 research planning、tension mining、critique 拆成獨立 stage。
custom agent / subagent	帶 persona、工具限制與 handoff 的角色封裝；description 會影響是否被委派。[S12][S18]	你要做 planner、scout、critic 等角色分工。	description 要寫得夠清楚，agent 才知道何時叫出它。

表 2-2 說明：這張表的核心觀點是『封裝層分工』。Project 比較像記憶與材料容器；`AGENTS.md` 是 always-on 規則；`prompt.md` 是單段流程模板；`SKILL.md` 與 custom agent 則是更高階、可按需觸發的能力包。只要你搞清楚每一層負責什麼，整個 agent 系統會明顯穩定很多。反過來說，若把所有東西都塞進 system prompt，最常見的結果就是冗長、互相干擾、且難以版本化。

這裡必須特別指出一個對你很關鍵的現實判斷。依官方說明，ChatGPT Pro 明確包含 Deep Research 與 Codex；但 Skills in ChatGPT 的 beta 可用對象，官方另有列出，並未在同一頁面中明確把個人版 Pro 納入同一敘述。[S3][S4][S9] 因此，對網頁版個人使用者，最穩妥的策略不是等待完整 skill UI，而是先用『Projects + Deep Research + 固定 pipeline prompts』模擬 skill；等流程成熟，再輸出成 `SKILL.md` 交給 Codex。

第 3 章 哪些 brainstorming 方法最適合得到新的發想

不是所有 brainstorming 方法都同樣適合做『新發想』。有些方法擅長重構問題，有些擅長窮舉變體，有些擅長處理矛盾，有些只適合補盲。若你想把它們做成 agent pipeline，方法的結構性尤其重要。

方法	在 pipeline 中的角色	新穎度潛力	結構化程度	是否容易 prompt 化
How Might We	重新框定問題，將限制轉成可探索的問題句。	中	中	高
SCAMPER	對既有方案做替換、組合、調整、用途轉換、刪減、反轉。	中高	高	很高
Morphological Analysis	把問題拆成多個維度，再做系統性組合與篩選。	很高	很高	很高
TRIZ	把工程或流程中的矛盾轉成發明原理與解法方向。	很高	高	中高
Six Thinking Hats	切換平行思考角色，避免過早爭辯與單視角。	中	高	高

Lateral Thinking	刻意改變概念與知覺路徑，打破線性延伸。	很高	中高	中高
Brainwriting	以沉默書寫方式大量產生候選點子，降低先發言者偏差。	中高	中	中
Worst Possible Idea	先生成糟糕答案，再反向回收可行創新。	高	中	高

表 3-1 說明：這張表不是方法學教科書，而是面向 agent 實作的比較。若某方法很難被 prompt 化，即使理論上優秀，也不一定適合先放進自動化流程。從這個角度看，Morphological Analysis、SCAMPER、TRIZ 的機械結構特別有價值，因為它們很容易被寫成明確步驟。

方法	最適合的使用時機	主要風險	我對你的優先級建議
How Might We	研究剛開始，題目還太大或太模糊時。	容易只改寫措辭，沒有真正改變問題空間。	中優先；放在最前面做 framing。
SCAMPER	你已經有一個 baseline 方法、產品或流程想做變體時。	容易產生看似很多、其實只屬微調的 ideas。	高優先；作為第一輪發散算子。
Morphological Analysis	你要找『以前沒人這樣組過』的方向時。	若維度定義得太差，會得到無意義組合。	最高優先；是 novelty 引擎。
TRIZ	你面對 trade-off、系統瓶頸、評測矛盾時。	需要先正確寫出矛盾，不然原理匹配會漂掉。	最高優先；尤其適合工程化 pipeline。
Six Thinking Hats	你需要補足風險、情感、創意、流程等不同視角時。	常被誤用成一般討論格式，而不是平行思考工具。	中優先；適合作為 critique 補盲層。
Lateral Thinking	你發現所有答案都太像，只是在既有框架內打轉時。	若沒有後續收斂，會停在荒誕概念。	最高優先；適合高新穎度回合。
Brainwriting	你想快速量產候選點子，並避免早期收斂時。	量多但品質參差，後續篩選成本高。	中高優先；適合作為早期 divergence。
Worst Possible Idea	模型過度保守，怎麼問都只給安全答案時。	若沒有反向轉譯，會只剩娛樂效果。	高優先；適合作為打破慣性的切換器。

表 3-2 說明：這張表把方法放回實際節點。對你要做的 agent workflow，我的建議是把 HMW 當成前處理、把 Morphological Analysis 與 TRIZ 視為核心 novelty engine、把 Lateral Thinking 與 Worst Possible Idea 視為突破僵局的高風險回合、再用 Six Thinking Hats 或 hostile reviewer 去補盲與刪除假創新。

方法排序總結 若目標是「真的得到新的研究方向」，我的實務排序如下。第一層是 Morphological Analysis、TRIZ、Lateral Thinking／Worst Possible Idea。第二層是 SCAMPER 與 Brainwriting。第三層是 HMW 與 Six Thinking Hats，前者偏 framing，後者偏補盲與結構化討論。這個排序不是在否定後兩者，而是指出它們更像支架，而不是 novelty engine 本身。

補充說明：TRIZ 與 LLM 的結合並非空想。近年的 AutoTRIZ 工作已開始展示如何將 TRIZ 的結構與大型語言模型結合，讓矛盾辨識與原理導向的 ideation 更容易被工作流化。[S26] 這強化了本報告的核心論點：你完全可以把本來看似『人類創意技巧』的方法，抽成 agent 可執行的 prompt pipeline。

第 4 章 從現行系統抽取得到的 prompt architecture

這一章回答最實用的問題：到底能不能從現行產品和方法裡抽出必要 prompt？答案是可以，而且應該抽的是多階段架構，而不是單一萬用 prompt。

階段	核心任務	輸入	輸出	停止條件
1. Framing	把模糊目標改寫成可研究、可評估、可驗證的問題。	主題、目的、限制、成功定義。	明確研究任務與新穎度標準。	問題句已足以指導來源選擇與評估。
2. Planning	決定來源範圍、子題、先廣後窄策略、交付 schema。	Framing 結果。	research plan。	計畫已可被審查與修改。
3. Landscape mapping	建立 field map、代表工作、資料與 benchmark 地圖。	research plan、來源。	場域地圖與子題分區。	主要子領域與來源結構已完整。
4. Tension mining	抽取矛盾、盲點、失敗模式、部署限制。	field map、來源摘要。	張力列表與機會空間。	不是只剩摘要，而是已找出可被設計的摩擦點。
5. Ideation operators	用 SCAMPER、Morphological、TRIZ、Lateral 等方法產生 idea seeds。	張力列表、假設清單、維度表。	候選 ideas。	至少產生足夠多且明顯不同構型的方向。
6. Hostile critique	刪除 trivial ideas，留下值得驗證的方向。	候選 ideas。	排序後的 top ideas 與風險分析。	每個保留項目皆有明確價值與最大風險。
7. Packaging	把有效流程升級為可重用 artifact。	最終工作流、成功案例。	`Project instructions`、 `.prompt.md`、 `AGENTS.md`、 `SKILL.md`。	流程已可被下次直接調用。

表 4-1 說明：這張表幾乎就是你之後要實作的總藍圖。它把聊天互動改寫成 protocol。真正重要的是最後一欄的停止條件；沒有停止條件，agent 很容易永遠停在收集更多資料，或永遠停在表面 brainstorm，而沒有產出可驗證方向。

若把官方產品與方法論一起看，可以抽出八個高價值原則。第一，先明確 outcome，再談搜尋。第二，先審查 research plan，再正式執行。第三，先廣後窄，不要一開始就鎖死在你最熟悉的子題。第四，把 research 與 ideation 明確拆開。第五，對所有候選想法強制區分 evidence-backed、inferred、speculative。第六，至少要有一輪 hostile critique。第七，把成功流程轉成 artifact。第八，把記憶層、流程層、封裝層分開存放。這八點分別來自 OpenAI、Anthropic、Google、VS Code 等系統的共同收斂。[S1][S2][S5][S6][S7][S10][S13][S17][S18]

在 ChatGPT Pro 與 Codex 中落地：記憶層、流程層、封裝層

這張圖說明什麼該放在 Project / OrionVault，什麼該放在 prompt pipeline，什麼該封裝成 AGENTS.md 或 SKILL.md。

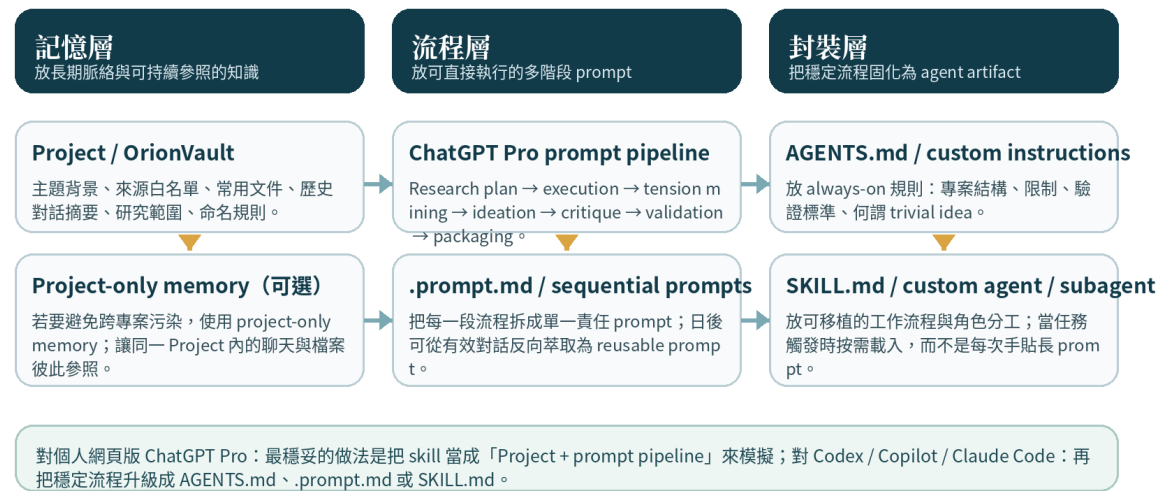


圖 4-1 在 ChatGPT Pro 與 Codex 中落地：記憶層、流程層、封裝層

圖 4-1 說明：這張圖回答一個經常混亂的問題：哪些資訊該放在 Project / OrionVault，哪些該寫成 prompt pipeline，哪些該升級成 `AGENTS.md` 或 `SKILL.md`。圖中的左欄是記憶層，存放長期脈絡與常參照資料，例如主題背景、來源白名單、命名規則、過去研究摘要；中欄是流程層，放可直接執行的多階段 prompts；右欄是封裝層，把穩定流程固化為 always-on 規則或可按需載入的 skill。這張圖最關鍵的訊息是：不要把記憶、流程、封裝塞在同一個位置。分層後，你的 agent 系統才會同時具有穩定性與可修改性。

第 5 章 給網頁版 ChatGPT Pro 的實作建議

對個人網頁版使用者，最可操作的答案不是等待完整的 skill UI，而是把 Project 當成記憶容器、把 Deep Research 當成研究引擎、把一串固定 prompts 當成流程層。這套組合已足以支撐高品質的 research-driven ideation。[S1][S2][S3][S4]

層	在 ChatGPT Pro 中的對應物	建議放入內容	不要放入的內容
記憶層	Project	主題背景、關鍵檔案、常用來源、命名規則、已知限制、過去決策摘要。	一次性任務細節、臨時試算、會很快失效的長 prompt。
研究層	Deep Research	帶限制的 research plan、指定網站、來源分層、下載報告。	高新穎度 brainstorming 本身。
流程層	連續 prompts	framing、planning、tension mining、ideation、critique、packaging。	always-on 規則與專案結構政策。
封裝層	固定 project instructions + 外部保存的模板	穩定成功的流程文字稿。	需要按 repo 結構與工具權限執行的工程規則。

表 5-1 說明：這張表給的是『個人網頁版最小可用方案』。重點不是模擬所有 agent 功能，而是先讓流程穩定運行。只要你能把檔案、研究、發想與批判分開，ChatGPT Pro 已足夠支撐非常強的工作流。

具體建議如下。第一步，在 Project 中放入專題背景、主要術語、核心文檔與來源白名單。第二步，先用 Deep Research 做 research plan，要求它不要直接產出 overview，而是建立 field map 並找 tensions。第三步，把研究結果帶回一般對話，用 Morphological Analysis、TRIZ 與 Lateral Thinking 重新生成 ideas。第四步，另開一輪 hostile critique，刪掉 trivial ideas。第五步，把這整套過程保存成可重用的 prompts。這一整串就是你在個人版 Pro 上的 skill 替代方案。

Pro 版的最實務理解

若你想最省力，請把『ChatGPT Pro 版 skill』理解為：Project 提供記憶與材料；Deep Research 提供研究與引用；一組固定順序的 prompts 提供可重跑的流程。也就是說，你先用追求名義上的 skill 介面，而是先把 skill 的本質做出來。

第 6 章 給 Codex / Copilot / Claude Code 的封裝建議

一旦某條工作流在 ChatGPT Pro 上穩定有效，就應考慮把它固化為 artifact。這時最重要的不是再把 prompt 加長，而是把責任邊界寫清楚。

artifact	負責什麼	推薦內容	不推薦內容
`AGENTS.md`	always-on 規則	專案結構、資料夾職責、輸出格式、驗證標準、命名規則、何謂 trivial idea、何時必須先提出 plan。[S7][S8]	長篇一次性任務細節、臨時想法。
`prompt.md`	單段流程模板	research planning、tension mining、Morphological pass、TRIZ pass、hostile critique 等單一責任 prompt。[S17]	整個專案的長期政策。
`SKILL.md`	可按需載入的完整能力包	goal、inputs、workflow、tool policy、output schema、stop conditions、failure recovery、novelty criteria。[S5][S6]	散亂備忘、沒有停止條件的閒聊式說明。
custom agent / subagent	角色分工與工具權限	planner、literature scout、critic、packager 等 description 與 handoff 條件。[S12][S18]	所有角色混在一起的超大 agent。

表 6-1 說明：這張表最重要的訊息是『先分責，再封裝』。如果在工作流還沒穩定之前就急著做 skill，最後做出來的通常是脆弱的長 prompt。相反地，先在網頁版或一般 chat 中找出穩定互動，再決定哪些東西應升級為 `AGENTS.md`、哪些應抽成 `prompt.md`、哪些值得變成 `SKILL.md`，成功率高得多。

對 Codex 而言，官方文件特別強調幾點：複雜任務先 plan，再執行；必要時先 interview 使用者以澄清目標；把反覆有效的模式寫進 `AGENTS.md`；若某能力值得跨專案重用，則再升級為 skill。[S5][S6][S7][S8] 這些建議與你要做的 brainstorming pipeline 非常契合，因為 ideation 本來就不該是一段無結構對話，而應是一條可審查的 research-to-idea protocol。

第 7 章 給 OrionVault / Obsidian 式記憶層的建議

你已經有 OrionVault 這種跨 agent 的記憶層，這反而是優勢。因為最難的不是模型本身，而是讓不同 agent 共享一致的背景、命名與歷史決策。下面這份結構是面向 Automatic Survey Generation 與 research-gap mining 任務而設計的。

資料夾	內容	更新頻率	供誰讀取
00_scope	主題定義、任務目的、輸出格式、排除範圍。	低	所有 agent 與人類使用者。
10_sources	來源白名單、黑名單、資料庫存取說明、關鍵網站。	中	research agent。
20_landscape	field map、代表方法、資料集、benchmark、近期趨勢。	高	research agent、planner。
30_tensions	矛盾、盲點、失敗模式、評測缺口、部署限制。	高	ideation agent、critic。
40_idea_seeds	尚未驗證的候選想法，按日期與主題整理。	高	ideation agent、PI 角色。
50_validated_directions	經過小驗證後保留的方向與其證據。	中	專案決策者、packager。
skills	`.SKILL.md`、角色說明、workflow artifacts。	中	Codex / agent runtime。
prompts	`.prompt.md`、聊天中萃取出 reusable prompts。	中	人與 agent 皆可。

表 7-1 說明：這份結構的關鍵，在於把『研究事實』與『發想產物』分開，把『可繼承記憶』與『可執行封裝』分開。最常見的失敗，是把未驗證想法、暫時筆記、正式結論、agent 規則全部寫在一起，結果任何 agent 都很難判斷哪些是背景、哪些是工作指令。

若你讓本地 Codex、Copilot、Gemini Code Assist 都以 OrionVault 當 system memory，那麼最值得優先穩定的是三種檔案。第一類是範圍與術語，避免不同 agent 對同一任務有不同理解。第二類是 tensions 與 blind spots，因為這是新發想的主要燃料。第三類是 skills 與 prompt files，因為這決定 agent 是否能重複執行成功流程。換句話說，Vault 不只是知識庫，更是 workflow registry。

附錄 A 可直接在 ChatGPT Pro 使用的完整 prompt pipeline

以下 prompts 以『可直接複製貼上』為目標設計。它們不是一次全部丟，而是依序執行。這樣做的理由是讓每一段都能被審查、修改與重跑。

A1 Project instructions

用途：先放進 Project 或在專案開頭當總規則。

本專案的目標不是做教科書式摘要，而是從研究與產品生態中萃取可執行的新方向。

固定工作流如下。

1. 先做 landscape mapping。
2. 再抽 tensions、contradictions、blind spots、failure modes。
3. 再做 structured ideation。
4. 再做 hostile critique。
5. 最後只保留可驗證且值得投入的方向。

每次輸出都必須明確區分：

- A. evidence-backed
- B. inferred
- C. speculative

優先尋找：

- A. 可從相鄰領域遷移的機制
- B. 彼此矛盾的結論或假設
- C. 評測與資料盲點
- D. 反覆出現但尚未被清楚命名的模式
- E. 需要新 workflow 才能解的瓶頸，而不是單純微調參數就能解的問題

框 A1 說明：這一段是最接近『個人版 skill』的東西。它不負責直接產出答案，而是規範整個專案如何判斷什麼算有價值的研究與發想。尤其是 evidence-backed/inferred/speculative 三分法，非常重要，因為它能防止 research 與 speculation 混成一團。

A2 Deep Research：先要計畫，不要立刻開跑

/Deepresearch

主題：<<topic>>

目標：不是寫 overview，而是找出能形成新方向的 idea seeds。

請先提出 research plan，不要立刻執行。

計畫必須包含：

1. 核心子領域。
2. 相鄰領域與可遷移領域。
3. 必查項目：contradictions、failure modes、evaluation blind spots、data blind spots、deployment constraints。
4. 來源分層：papers、official docs、products、blogs、benchmarks。
5. 判定「新發想」的標準。
6. 最終輸出 schema：field map、tensions、underexplored combinations、idea seeds、falsification checks。

框 A2 說明：這一段直接對應官方 Deep Research 的 research plan 介面。重點不是 prompt 很長，而是要求『先給計畫』。這樣你可以在研究真正開始前修正來源與輸出格式，避免系統直接把時間花在你不需要的方向上。

A3 Deep Research：正式執行

依照剛才的計畫執行 deep research。

要求：

1. 先 breadth-first，再 narrow down。
2. 不要一開始就鎖死單一子題。
3. 每個重要 claim 都要能回到來源。
4. 對每個候選方向都區分：
 - A. 已有共識
 - B. 有證據但尚未被整合命名
 - C. 高潛力但推測性強

最後請輸出：

- A. field map
- B. tensions / contradictions
- C. underexplored combinations
- D. 10 至 15 個 idea seeds
- E. 每個 seed 的 supporting evidence 與 biggest risk

框 A3 說明：這一段最重要的設計點有兩個。第一是『先廣後窄』，這是 Anthropic 明確公開的 research heuristic，也非常適合避免你在一開始就被單一路徑綁死。第二是最後的輸出 schema 已經替後段 ideation 做準備，尤其是 tensions 與 underexplored combinations 兩欄。

A4 把 research 結果轉成 ideas

不要再做摘要。請把目前研究結果轉成發想。

請做三輪：

第一輪：SCAMPER

第二輪：Morphological Analysis

第三輪：Cross-domain transfer

要求：

1. Morphological Analysis 至少選 5 個維度，例如：
problem unit、data source、model family、evaluation regime、deployment constraint
2. Cross-domain transfer 至少從 3 個相鄰領域各借 2 個機制
3. 總共輸出 20 個 idea seeds

每個 idea seed 固定 5 行：

1. 核心概念
2. 它重組了哪些既有元素
3. 為何現在可能可行
4. 最小驗證
5. 最大風險

框 A4 說明：這一段是整套工作流的 novelty pivot。它刻意要求先做 SCAMPER，再做 Morphological Analysis，再做跨領域遷移，原因是三者的思考模式不同。SCAMPER 讓你先快速變形，Morphological Analysis 強迫你做系統組合，跨領域遷移則把相鄰領域機制拉進來，三者疊加後，點子會比一般 brainstorm 明顯更分散。

A5 高新穎度回合

現在切到高新穎度模式。

請用 TRIZ + Lateral Thinking + Worst Possible Idea 重新生成一次候選方向，但只保留真的跳脫既有框架的 ideas。

步驟：

1. 先列出這個領域最常見的 5 個隱含假設。
2. 對每個假設做反轉、替換、移位或矛盾化。
3. 產生 10 個高風險高新穎度方向。
4. 每個方向都標記：
 - A. 它違反了哪個既有假設
 - B. 它可能解決什麼新問題
 - C. 它最可能在哪裡失敗

框 A5 說明：若模型一直給出太安全的答案，這一段通常是最有效的切換器。TRIZ 負責把矛盾說清楚，Lateral Thinking 負責改變概念路徑，Worst Possible Idea 則專門用來突破保守傾向。三者合用的目的不是追求荒誕，而是強迫系統離開熟悉局部最優解。

A6 刪掉假創新

現在扮演 hostile reviewer + pragmatic PI。

對前面所有 ideas 做四維評估：

- A. novelty
- B. plausibility
- C. evidence support
- D. execution cost

請刪掉以下類型：

1. 只是改名
2. 只是參數微調
3. 只是把舊方法搬到新 domain 但沒有新 interaction
4. 只是把多個模組拼在一起但沒有新假設或新 evaluation value

最後只留下 3 至 5 個方向，並為每個方向輸出：

1. 為什麼值得做
2. 30 天內的最小驗證
3. 需要的資料、工具、協作者
4. 最可能失敗的原因
5. 失敗後的 pivot 路徑

框 A6 說明：這一段是整份報告最務實的一段。大部分 brainstorming 的失敗，不是因為沒有點子，而是沒有把『看似新、其實只是換皮』的東西刪掉。這段 prompt 把 hostile reviewer 與 pragmatic PI 合併，目的就是同時考慮學術新穎度與執行現實。

A7 把成功流程封裝回去

把本次最有效的工作流整理成兩種版本：

1. 給人看的 research brief
2. 給 agent 跑的 workflow brief

workflow brief 必須包含：

- A. goal
- B. inputs
- C. step sequence
- D. tool use policy
- E. output schema
- F. stop conditions
- G. failure recovery
- H. criteria for novelty vs triviality

框 A7 說明：很多人做到這裡就停，但真正的生產力來自封裝。只要你把本次有效流程重新輸出成 workflow brief，下次就不必從零開始，而是可以逐步升級成 `prompt.md`、`AGENTS.md` 或 `SKILL.md`。

A8 面向 Automatic Survey Generation 的專用轉譯

請把上面的 top ideas 改寫成面向 Automatic Survey Generation 的版本。

要求：

1. 每個方向要明確指出它對 survey pipeline 的哪一段有影響。
2. 每個方向至少提出一個可觀察的 research gap mining 訊號。
3. 每個方向要列出最適合納入 OrionVault 的持久化欄位。
4. 最後請輸出一張表：idea、evidence、gap signal、minimum validation、vault fields。

框 A8 說明：這是依你的專案場景客製化的一段。它的重點不是再創造新方法，而是把前面抽出的 ideas 重新投影到 survey generation pipeline 與 OrionVault 結構上，讓產物更容易直接落地。

附錄 B 單次使用的濃縮版總 prompt

若你暫時不想跑整套 pipeline，可以用下面這個濃縮版。但要知道，濃縮版在新穎度與可控性上通常不如分段版。

B1 濃縮版總 prompt

你現在不是一般聊天助手，而是一個 research-driven ideation engine。你的任務不是寫 overview，而是從多來源研究中找出具有新穎度、可驗證、可封裝為 agent workflow 的方向。

請依序完成：

1. 定義問題與成功標準。
2. 建立 field map，並列出主要子領域、代表方法、資料、benchmark 與部署情境。
3. 抽取 tensions、contradictions、failure modes、evaluation blind spots、data blind spots。
4. 用 SCAMPER、Morphological Analysis、TRIZ、Lateral Thinking、Worst Possible Idea 生成 ideas。
5. 刪除 trivial ideas，只保留真正值得驗證的 3 至 5 個方向。
6. 為每個方向提供：核心概念、支持證據、為何現在可行、30 天內最小驗證、最大風險、失敗後 pivot。
7. 最後把有效流程輸出成：

- A. Project instructions
- B. 一組 sequential prompts
- C. `AGENTS.md` 摘要
- D. `SKILL.md` 摘要

全程請明確區分 evidence-backed、inferred、speculative；並在需要時指出哪些內容只是推測。

框 B1 說明：這個版本適合快速探索，但不適合正式沉澱 workflow。因為它把多個階段壓在一起，導致你很難審查每一步的品質，也比較難在中間調整研究方向。

附錄 C `AGENTS.md` 模板

下面這份模板的目的，是把專案級 always-on 規則寫清楚。它不代替 workflow prompt，而是規範 agent 平常做事的方式。

C1 `AGENTS.md` 範本

```
# AGENTS.md

## Purpose
This repository supports research-driven ideation for topic discovery, survey generation, and workflow packaging.

## Core behavior
1. Plan before execution for any non-trivial task.
2. Separate research, ideation, critique, and packaging into distinct steps.
3. Distinguish clearly between evidence-backed findings, inferred patterns, and speculative proposals.
4. Never present rename-only, tuning-only, or shallow-transfer ideas as meaningful novelty.
5. When uncertain, propose a short research plan instead of pretending certainty.

## Repository structure
- 00_scope: task definition and exclusions
- 10_sources: source policies and whitelists
- 20_landscape: field maps and domain summaries
- 30_tensions: contradictions, blind spots, failure modes
- 40_idea_seeds: unvalidated ideas
- 50_validated_directions: ideas surviving critique or pilot validation
- prompts: reusable stage prompts
- skills: portable workflow artifacts

## Output rules
1. Prefer structured outputs over long prose when comparing candidates.
2. Every retained idea must include:
  - why it matters
  - supporting evidence
  - minimum validation
  - biggest risk
```

3. Every research summary must include:

- scope
- sources consulted
- main tensions
- open uncertainties

Novelty filters

Reject ideas that are:

1. rename-only
2. scale-only
3. parameter-tuning-only
4. shallow domain transfer without new interaction or evaluation value
5. module-stacking without a new assumption, mechanism, or validation target

When to escalate

1. If a workflow works repeatedly, propose extracting it into `.prompt.md`.
2. If a capability is reusable across tasks, propose packaging it as `SKILL.md`.
3. If a task is ambiguous, ask 3 focused clarification questions or present a draft plan first.

框 C1 說明：這份模板的風格刻意偏短、偏規則化。因為 `AGENTS.md` 的任務不是解釋背景，而是提供平時始終生效的操作規則。若把單次任務細節塞進去，之後很容易與其他任務互相污染。

附錄 D `SKILL.md` 模板

下面這份模板面向需要按需載入的可攜 workflow。它比 `AGENTS.md` 更像一個能力包。

D1 `SKILL.md` 範本

```
---
name: research-ideation
description: Generate novel, defensible directions from a topic by mapping the landscape, mining tensions,
applying structured ideation operators, and stress-testing the outputs.
---

## Objective
Produce new, defensible idea directions rather than generic summaries.

## Inputs
- topic
- scope and exclusions
- preferred sources or source whitelist
- success criteria for novelty
- optional repository paths for output persistence

## Workflow
1. Clarify the target outcome.
2. Propose a research plan before execution.
3. Build a field map:
```

- core subareas
 - adjacent fields
 - datasets and benchmarks
 - deployment contexts
4. Extract tensions:
- conflicting findings
 - missing evaluations
 - data blind spots
 - operational bottlenecks
5. Generate ideas using:
- SCAMPER
 - Morphological Analysis
 - TRIZ or contradiction-based reasoning
 - Lateral reversal / Worst Possible Idea
6. Separate:
- evidence-backed findings
 - inferred patterns
 - speculative proposals
7. Kill weak ideas:
- rename-only
 - tuning-only
 - shallow transfer
 - module stacking without new value
8. Return:
- field map
 - tensions
 - 15 idea seeds
 - ranked top ideas
 - 30-day validation plan
 - packaging recommendations

Output schema

- scope
- field_map
- tensions
- idea_seeds
- shortlisted_ideas
- validation_plan
- packaging_artifacts

Stop conditions

- Research is broad enough to cover the main subareas and adjacent fields.
- Tensions are explicit enough to support ideation.
- Final ideas include both support and risk.
- At least one critique pass has been completed.

Failure recovery

- If the topic is too broad, narrow the scope with a revised framing prompt.

- If ideas are too generic, rerun the ideation stage with stronger contradiction and lateral prompts.
- If evidence is weak, return to research with targeted source expansion.

框 D1 說明：`SKILL.md` 的價值在於按需載入。它不像 system prompt 那樣永遠跟著 agent，而是由名稱與 description 先被檢索，相關時再讀入完整 workflow。這就是為什麼你要把 objective、inputs、workflow、stop conditions 寫得清楚。

附錄 E 常見失敗模式與修正策略

失敗模式	症狀	根本原因	修正策略
只有漂亮摘要，沒有新方向	輸出很完整，但所有 ideas 都像文獻回顧的延長。	把 research 與 ideation 混成同一步。	強制在 research 後獨立執行 Morphological / TRIZ / Lateral 回合。
ideas 看起來很多，其實都在微調	二十個點子彼此高度相似。	只用了 SCAMPER 或自由 brainstorm，沒有組合爆炸與假設翻轉。	加入 Morphological Analysis 與 Worst Possible Idea。
模型一直給太安全的答案	沒有任何方向敢挑戰領域常識。	缺少高風險回合與 hostile critique 前的發散。	顯式列出隱含假設，逐一反轉或矛盾化。
流程難以重複	每次都得重新解釋背景。	沒有把記憶層與流程層分開。	用 Project / OrionVault 存背景，用 prompts 存流程。
agent 讀很多卻不會停	一直想再搜更多資料。	沒有停止條件與 output schema。	對每一階段加 stop conditions。
封裝後效果變差	原本聊天有效，做成 skill 反而僵硬。	過早封裝，或把多個責任硬塞進同一 artifact。	先找穩定互動，再拆成 `AGENTS.md`、`.prompt.md`、`SKILL.md`。

表 E-1 說明：這張表是從實作角度整理的錯誤清單。它最有價值的地方不是指出錯誤，而是指出每種錯誤通常是由哪一層設計缺失造成的。只要你把『沒有新方向』這件事分解為研究、發想、批判、封裝四段，問題通常都能定位。

結語

若把本報告濃縮成最實務的建議，就是這四句話。第一，不要把 Deep Research 誤認為 brainstorming 本身。第二，真正能逼出新方向的，通常是 Morphological Analysis、TRIZ、Lateral Thinking／Worst Possible Idea 這類後段算子。第三，對網頁版 ChatGPT Pro，先用 Projects + Deep Research + prompt pipeline 模擬 skill。第四，等流程穩定後，再把它升級成 `AGENTS.md`、`.prompt.md`、`SKILL.md` 交給 Codex 與其他 agent。

換句話說，你要建的不是一個魔法 prompt，而是一條可反覆執行、可批判、可驗證、可封裝的研究到發想 workflow。這正是目前主流系統與方法論共同收斂出的方向。

參考來源

以下來源為本報告撰寫時所依據的主要官方文件、方法論說明與補充研究。

- S1 OpenAI Help Center, Deep research in ChatGPT。用途：用於界定 Deep Research 的控制項、來源策略、研究計畫與下載能力。
- S2 OpenAI Help Center, Projects in ChatGPT。用途：用於界定 Projects 的工作空間、檔案、指令與記憶行為。
- S3 OpenAI Help Center, About ChatGPT Pro plans。用途：用於確認 Pro 方案包含 Codex、Deep Research 等功能。
- S4 OpenAI Help Center, Skills in ChatGPT。用途：用於界定 ChatGPT Skills 的 beta 可用性與與 Codex/API 的關係。
- S5 OpenAI Developers, Agent Skills – Codex。用途：用於整理 Codex skills 的設計原則與實作建議。
- S6 OpenAI API Docs, Skills guide。用途：用於整理技能掛載、觸發與 `SKILL.md` 的載入方式。
- S7 OpenAI Developers, Best practices – Codex。用途：用於整理 plan-first、interview-first、將有效流程升級為 artifact 的做法。
- S8 OpenAI Developers, Custom instructions with AGENTS.md。用途：用於界定 `AGENTS.md` 的 always-on 行為。
- S9 OpenAI Help Center, Using Codex with your ChatGPT plan。用途：用於確認 Codex 可在 ChatGPT 訂閱方案內使用。
- S10 Anthropic Engineering, How we built our multi-agent research system。用途：用於整理 orchestrator-worker、start wide then narrow、parallel subagents、judge rubric 等 heuristic。
- S11 Claude Help Center, Using Research on Claude。用途：用於比較 Claude Research 的 citations 與 agentic 多輪搜尋特性。
- S12 Claude Code Docs, Create custom subagents。用途：用於說明以 description 觸發 subagent 委派的機制。
- S13 Gemini Apps Help, Use Deep Research。用途：用於比較 Deep Research 的可編輯 research plan 與來源選擇。
- S14 Google AI for Developers, Gemini Deep Research Agent。用途：用於說明 API 層的多步研究代理。
- S15 Perplexity Help Center, What's New in Advanced Deep Research。用途：用於比較高召回研究、進度追蹤與可編輯報告。
- S16 VS Code Docs, Use custom instructions in VS Code。用途：用於對照 repo 級 always-on 指令。
- S17 VS Code Docs, Use prompt files in VS Code。用途：用於說明 `.prompt.md` 的定位，以及從對話抽取 reusable prompt。
- S18 VS Code Docs, Custom agents in VS Code。用途：用於說明 custom agent、handoff 與受限工具。
- S19 Stanford d.school, How Might We Questions。用途：用於界定 HMW 是重新框定問題的 ideation scaffolding。
- S20 Interaction Design Foundation, SCAMPER。用途：用於整理 SCAMPER 七種變形操作。
- S21 Tom Ritchey / Swedish Morphological Society, Morphological Analysis。用途：用於界定形態分析是多維、非量化問題的系統組合方法。
- S22 The de Bono Group, Six Thinking Hats。用途：用於界定平行思考與角色切換。
- S23 Interaction Design Foundation, Brainwriting。用途：用於整理沉默書寫式發想的特性。
- S24 Interaction Design Foundation, Worst Possible Idea。用途：用於整理反向發想與破除假設的技巧。
- S25 The de Bono Group, Lateral Thinking。用途：用於界定 lateral thinking 是刻意、系統化地以不同方式思考。
- S26 Jiang & Luo, AutoTRIZ: Artificial Ideation with TRIZ and Large Language Models。用途：用於補充 TRIZ 已開始被 LLM 自動化，說明 prompt 化與工作流化的可行性。

使用提醒

產品功能與方案可用性會持續變動，因此實作時仍建議以最新官方文件為準。本報告的真正價值，不在於某一家產品的單一功能，而在於抽出一個可跨平台遷移的 prompt architecture：先研究、再抽張力、再做結構化發想、再惡意審稿、最後封裝。